

```
1 using MinecraftUpdateAndSync.Models;
2 using System;
3 using System.Collections.Generic;
4 using System.Linq;
5 using System.Text;
6 using System.Threading.Tasks;
7 using MinecraftUpdateAndSync.Utilities;
8 using System.Net.Security;
9 using System.Security.Cryptography.X509Certificates;
10
11 namespace MinecraftUpdateAndSync.Services
12 {
13     public class CompareService
14     {
15         public class CompareResult
16         {
17             public string RelativePath { get; set; }
18             public FileState State { get; set; }
19             public SyncAction Action { get; set; }
20         }
21
22         private static FileState CompareHash(string hash1, string hash2)
23         {
24             if (string.IsNullOrEmpty(hash1))
25             {
26                 LogHelper.LogWarning($"Hash1 is null.",
27                                     LogHelper.LogDebugLevel.Low);
28                 return FileState.InvalidLocal;
29             }
30             if (string.IsNullOrEmpty(hash2))
31             {
32                 LogHelper.LogWarning($"Hash2 is null.",
33                                     LogHelper.LogDebugLevel.Low);
34                 return FileState.InvalidRemote;
35             }
36             if (hash1.Equals(hash2, StringComparison.OrdinalIgnoreCase))
37             {
38                 return FileState.Same;
39             }
40             return FileState.Different;
41         }
42
43         // Temporay not used, as we will compare file hash for MVP. We can
44         // consider to use file mtime for quick compare in the future, and
45         // only compare file hash when mtime is different.
46         private static FileState CompareMTime(DateTime mTime1, DateTime
47         mTime2)
48         {
49             if (mTime1.CompareTo(mTime2) == 0)
50             {
51                 return FileState.Same;
52             }
53             return FileState.Different;
54         }
55
56         // We assume that the file name is the same, so there is no need to
```

```
        compare the file name.
52    // Action will be handled in the other functions.
53    public static CompareResult CompareFile(FileSnapshot localFile,           ↗
        ManifestFile remoteFile)
54    {
55        if (localFile == null || remoteFile == null)
56        {
57            LogHelper.LogWarning($"One or both file snapshots are null.", ↗
                LogHelper.LogDebugLevel.None);
58            return null;
59        }
60        var result = new CompareResult
61        {
62            RelativePath = remoteFile.RelativePath,
63        };
64        if (string.IsNullOrEmpty(localFile.FullPath) || !                ↗
            System.IO.File.Exists(localFile.FullPath))
65        {
66            LogHelper.LogError($"Local file is incomplete.",                ↗
                LogHelper.LogDebugLevel.None);
67            result.State = FileState.InvalidLocal;
68            return result;
69        }
70        if (string.IsNullOrEmpty(remoteFile.RelativePath) ||                ↗
            string.IsNullOrEmpty(remoteFile.Hash) || remoteFile?.MTime == ↗
            null)
71        {
72            LogHelper.LogError($"Remote file is incomplete.",                ↗
                LogHelper.LogDebugLevel.None);
73            result.State = FileState.InvalidRemote;
74            return result;
75        }
76        // compare file hash only for MVP; we can consider to compare        ↗
            file mtime for quick compare in the future, and only compare        ↗
            file hash when mtime is different.
77        if (string.IsNullOrEmpty(localFile.Hash) &&                ↗
            HashHelper.TryComputeFileHash(localFile.FullPath, out string ↗
            hash1))
78        {
79            localFile.Hash = hash1;    // Cache the computed hash to        ↗
                avoid recomputing it in the future if needed.
80            LogHelper.LogInfo($"Local file hash is computed: \nFile path: ↗
                {localFile.FullPath}\nHash: {localFile.Hash}",                ↗
                LogHelper.LogDebugLevel.High);
81        }
82        result.State = CompareHash(localFile.Hash, remoteFile.Hash);
83        LogHelper.LogInfo($"Local file hash compared with remote file        ↗
            hash: \n" +
84            $"Local file path: {localFile.FullPath}\n" +
85            $"Remote file path: {remoteFile.RelativePath}\n" +
86            $"Local file hash: {localFile.Hash}\n" +
87            $"Remote file hash: {remoteFile.Hash}\n" +
88            $"Result: {result.State}", LogHelper.LogDebugLevel.High);
89        return result;
90    }
91
```

```
92     private static SyncAction GetSyncAction(FileState state)
93     {
94         switch (state)
95         {
96             case FileState.Same:
97                 return SyncAction.None;
98             case FileState.MissingLocal:
99                 return SyncAction.Download;
100             case FileState.MissingRemote:
101                 return SyncAction.Delete;
102             case FileState.Different:
103                 return SyncAction.Replace;
104             case FileState.InvalidLocal:
105                 return SyncAction.Replace;
106             case FileState.Ignored:
107                 return SyncAction.None;
108             case FileState.InvalidRemote:
109                 return SyncAction.Ignore;
110             default:
111                 return SyncAction.None;
112         }
113     }
114
115     public static List<CompareResult> CompareManifestToDictLocalFiles(
116         Manifest manifest,
117         Dictionary<string, FileSnapshot> localFiles)
118     {
119         var compareResults = new List<CompareResult>();
120         foreach (var remoteFile in manifest.Files)
121         {
122             if (localFiles.TryGetValue(remoteFile.RelativePath, out FileSnapshot localFile))
123             {
124                 var result = CompareFile(localFile, remoteFile);
125                 if (result != null)
126                 {
127                     result.Action = GetSyncAction(result.State);
128                     compareResults.Add(result);
129                 }
130                 else
131                 {
132                     LogHelper.LogError($"Compare file result is null for
133                                     file, skipping: {remoteFile.RelativePath}",
134                                     LogHelper.LogDebugLevel.None);
135                 }
136             }
137             else
138             {
139                 // Local file is missing.
140                 var result = new CompareResult
141                 {
142                     RelativePath = remoteFile.RelativePath,
143                     State = FileState.MissingLocal,
144                     Action = GetSyncAction(FileState.MissingLocal)
145                 };
146                 compareResults.Add(result);
147             }
148         }
149     }
```

```
145     }
146 }
147     return compareResults;
148 }
149
150 public static List<CompareResult> CompareDeletedManifestToDictLocalFiles(
151     Manifest deletedManifest,
152     Dictionary<string, FileSnapshot> localFiles)
153 {
154     var compareResults = new List<CompareResult>();
155     foreach (var remoteFile in deletedManifest.Files)
156     {
157         if (localFiles.TryGetValue(remoteFile.RelativePath, out FileSnapshot localFile))
158         {
159             // Local file exists but it should be deleted.
160             var result = new CompareResult
161             {
162                 RelativePath = remoteFile.RelativePath,
163                 State = FileState.MissingRemote,
164                 Action = GetSyncAction(FileState.MissingRemote)
165             };
166             compareResults.Add(result);
167         }
168         else
169         {
170             // Local file is missing, which is expected for deleted
171             // files, so we can ignore this case.
172             LogHelper.LogInfo($"Local file is already missing for
173                 deleted file: {remoteFile.RelativePath}",
174                 LogHelper.LogDebugLevel.Low);
175         }
176     }
177     return compareResults;
178 }
179
180 /// <summary>
181 /// Compares two manifests and determines the differences between
182 /// their files, including added, removed, and
183 /// changed files.
184 /// </summary>
185 /// <remarks>The comparison treats the manifest with the higher
186 /// version as the source of truth.
187 /// The results indicate which files are unchanged, need to be
188 /// replaced, downloaded, or deleted to synchronize
189 /// the manifests.</remarks>
190 /// <param name="manifest1">The first manifest to compare. Represents
191 /// one version of the file set.</param>
192 /// <param name="manifest2">The second manifest to compare.
193 /// Represents the other version of the file set.</param>
194 /// <returns>A list of compare results indicating the state and
195 /// required synchronization action for each file. Returns
196 /// null if the manifest versions are equal.</returns>
197 public static List<CompareResult> CompareManifests(Manifest
198     manifest1, Manifest manifest2)
```

```
189     {
190         var compareResults = new List<CompareResult>();
191         var manifest1Version = manifest1.Version;
192         var manifest2Version = manifest2.Version;
193         Dictionary<string, ManifestFile> oldManifestDict = null;
194         Dictionary<string, ManifestFile> newManifestDict = null;
195         // newer as the target (manifest2 as target)
196         if (Manifest.CompareVersions(manifest1Version,           ↗
197                                     manifest2Version) // manifest1Version is greater than ↗
198                                     manifest2Version
199     {
200         oldManifestDict = manifest2.Files.ToDictionary(f =>           ↗
201                                                         f.RelativePath, f => f);
202         newManifestDict = manifest1.Files.ToDictionary(f =>           ↗
203                                                         f.RelativePath, f => f);
204     }
205     else if (Manifest.CompareVersions(manifest2Version,           ↗
206                                     manifest1Version) // manifest2Version is greater than ↗
207                                     manifest1Version
208     {
209         oldManifestDict = manifest1.Files.ToDictionary(f =>           ↗
210                                                         f.RelativePath, f => f);
211         newManifestDict = manifest2.Files.ToDictionary(f =>           ↗
212                                                         f.RelativePath, f => f);
213     }
214     else // manifest1Version is equal to manifest2Version
215     {
216         return new List<CompareResult>();
217     }
218     foreach (var newFile in newManifestDict.Values)
219     {
220         var result = new CompareResult();
221         if (oldManifestDict.TryGetValue(newFile.RelativePath, out ↗
222                                         ManifestFile manifest2File))
223         {
224             // The file exists in both manifests, compare the file ↗
225             hash to see if the file is changed.
226             if (CompareHash(newFile.Hash, manifest2File.Hash) == ↗
227                 FileState.Same)
228             {
229                 result.RelativePath = newFile.RelativePath;
230                 result.State = FileState.Same;
231                 result.Action = GetSyncAction(result.State);
232             }
233             else
234             {
235                 result.RelativePath = newFile.RelativePath;
236                 result.State = FileState.Different;
237                 result.Action = GetSyncAction(result.State); // The ↗
238                 file is different, we need to replace the old file ↗
239                 with the new file.
240             }
241         }
242     }
243     else
244     {
245     }
```

```
232         // The file is missing in local manifest
233         result.RelativePath = newFile.RelativePath;
234         result.State = FileState.MissingLocal;
235         result.Action = GetSyncAction(result.State);
236     }
237     compareResults.Add(result);
238 }
239 foreach (var oldFile in oldManifestDict.Values)
240 {
241     if (!newManifestDict.TryGetValue(oldFile.RelativePath, out ManifestFile manifest2File)) ?
242     {
243         // The file is missing in remote manifest, which means ?
244         // the file is deleted in the newer manifest, we need to ?
245         // delete the local file.
246         var result = new CompareResult
247         {
248             RelativePath = oldFile.RelativePath,
249             State = FileState.MissingRemote,
250             Action = GetSyncAction(FileState.MissingRemote)
251         };
252         compareResults.Add(result);
253     }
254 }
255 return compareResults;
256 }
257 }
```